جامعة السلطان مولاي سليمان
Université Sultan Moulay Slimane

المدرسة الوطنية للعلوم التطبيقية - بني ملال
Ecole Nationale des Sciences Appliquées - Béni Mellal

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
Béni Mellal — Maroc

TP 1

Application Monolithique avec Spring Boot

Gestion de Produits Simple

Réalisé par :

Diallo Thierno Mohammad Lamine

Encadrant :

Pr. BE ELBAGHAZAOUI

1. Introduction et Objectifs

Ce travail pratique a pour but de concevoir et de mettre en œuvre une application web monolithique à l'aide du framework Spring Boot. L'application réalisée permet la gestion d'un catalogue de produits e-commerce au travers d'une API REST complète respectant les standards HTTP.

Un monolithe est une architecture dans laquelle l'ensemble des fonctionnalités de l'application (interface, logique métier, accès aux données) est regroupé au sein d'un seul et unique déploiement. Cette approche est particulièrement adaptée aux projets de petite et moyenne taille, car elle simplifie le développement, le débogage et la mise en production.

Objectifs du TP

- Créer une application Spring Boot monolithique structurée en couches.
- Développer une API REST CRUD complète (GET, POST, PUT, DELETE).
- Comprendre l'architecture en couches : Entity, Repository, Service, Controller.
- Utiliser @RepositoryRestResource pour générer automatiquement des endpoints REST.
- Réaliser l'exercice d'extension : ajout de l'entité Category avec relation @ManyToOne.
- Tester l'ensemble des fonctionnalités via Postman.

Outils et Technologies utilisés

Outil / Technologie	Version	Rôle
Java / JDK	17+	Langage principal
Spring Boot	3.x	Framework applicatif
Spring Data JPA	3.x	Accès aux données
Spring Data REST	3.x	API REST automatique
PostgreSQL / MySQL	—	Base de données
Lombok	—	Réduction du boilerplate
IntelliJ IDEA	—	Environnement de développement
Postman	—	Tests des endpoints REST
Maven	3.8+	Gestion des dépendances

2. Architecture du Projet

L'application adopte une architecture en couches (Layered Architecture), principe fondamental du développement Spring Boot. Chaque couche possède une responsabilité unique et bien délimitée, favorisant la maintenabilité et la testabilité du code.

Les 4 couches de l'application

Couche	Classe principale	Rôle
Entité (Model)	Product.java	Représente la table SQL "products", portant les annotations JPA et les validations Jakarta.
Repository	ProductRepository.java	Interface étendant JpaRepository, fournissant les opérations CRUD. Utilise @RepositoryRestResource pour exposer une API automatique.
Service	ProductService.java	Contient la logique métier. Gère les transactions avec @Transactional et délègue la persistance au Repository.
Contrôleur	ProductController.java	Expose les endpoints REST sur /api/products. Reçoit les requêtes HTTP et retourne des ResponseEntity typées.

Cette organisation respecte le principe de séparation des préoccupations (Separation of Concerns) : le contrôleur ne connaît pas la base de données, le service ne connaît pas HTTP, et le repository n'applique aucune logique métier.

Structure du package Java

```
com.ecommerce.monolith └── product      ├── model      │    ├── Product.java      │
└── Category.java      ├── repository      │    ├── ProductRepository.java      │
└── CategoryRepository.java      ├── service      │    └── ProductService.java
└── controller          └── ProductController.java
```

3. Tests des Endpoints REST avec Postman

L'ensemble des endpoints REST a été testé avec Postman. L'application est démarrée localement sur le port 8080. Voici les résultats obtenus pour chaque opération CRUD.

3.1 GET — Liste de tous les produits

La requête GET sur /api/products retourne la liste complète des produits enregistrés en base. Le code de réponse est 200 OK.

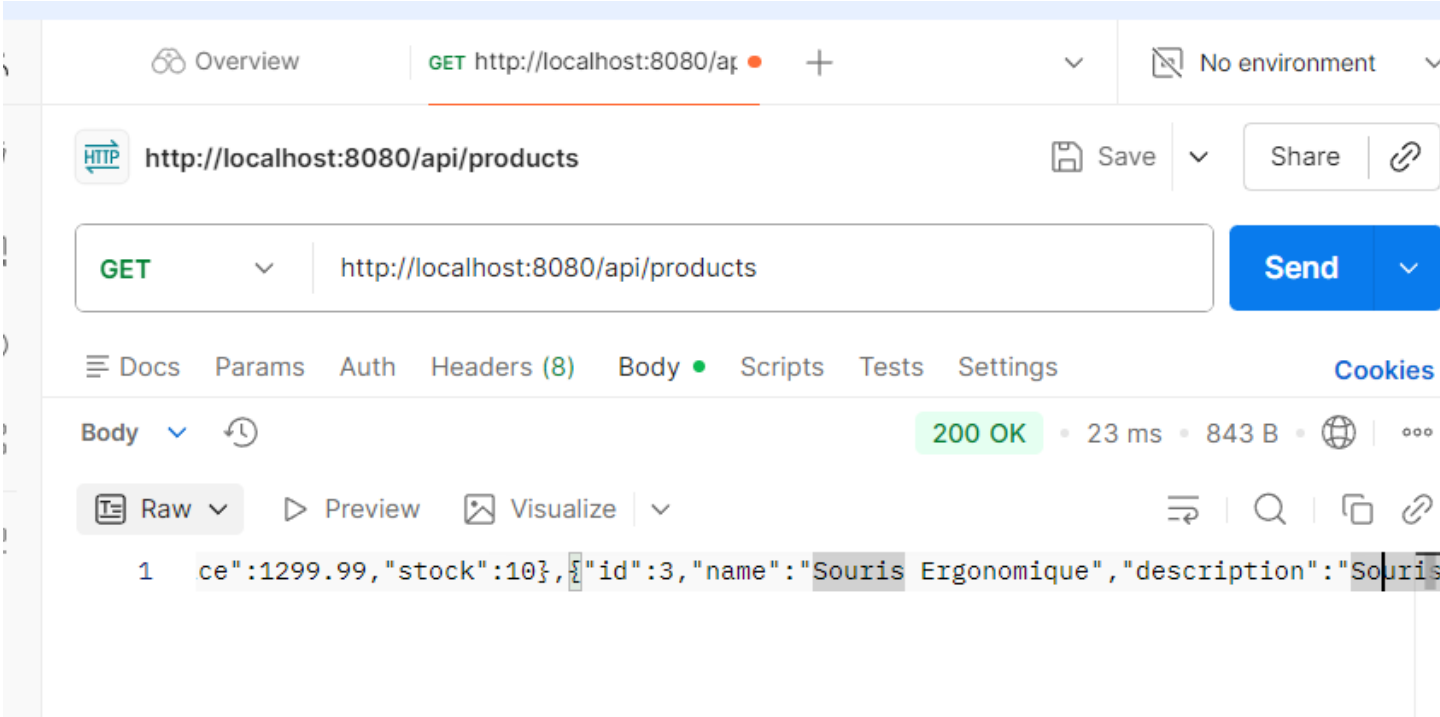

Figure 1 — GET /api/products : réponse 200 OK avec la liste des produits

3.2 POST — Création d'un produit

La requête POST sur /api/products avec un corps JSON permet de créer un nouveau produit. Dans cet exemple, l'iPhone 14 Pro Max est créé avec un prix de 2300,50 et un stock de 5. La réponse attendue est 201 Created.

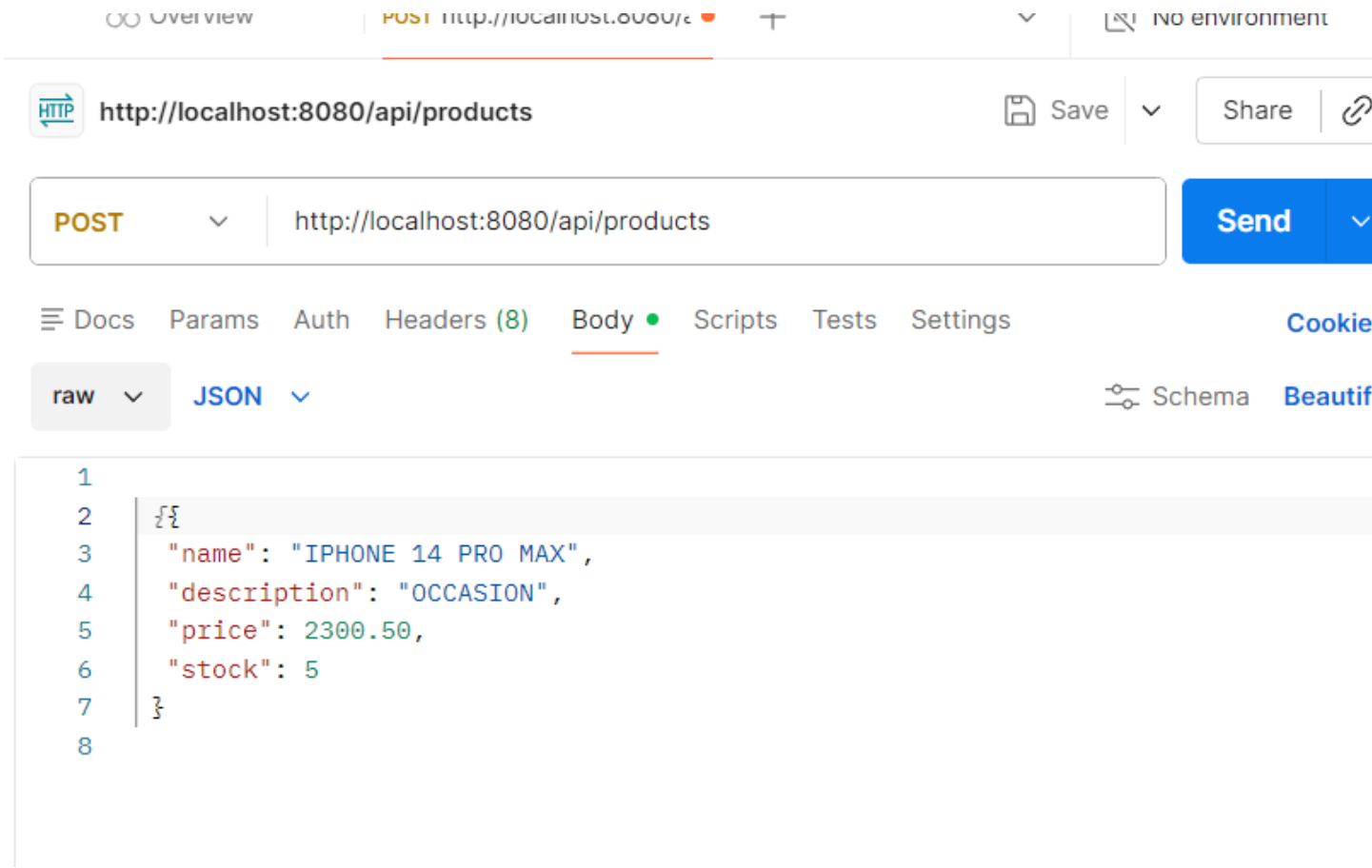

Figure 2 — POST /api/products : création d'un produit (iPhone 14 Pro Max)

3.3 PUT — Mise à jour d'un produit

La requête PUT sur /api/products/{id} permet de mettre à jour intégralement un produit existant. Ici, le produit d'ID 7 est mis à jour avec le nom "SAMSUNG", un prix de 8300,50 et un stock de 2. La réponse est 200 OK avec les données mises à jour.

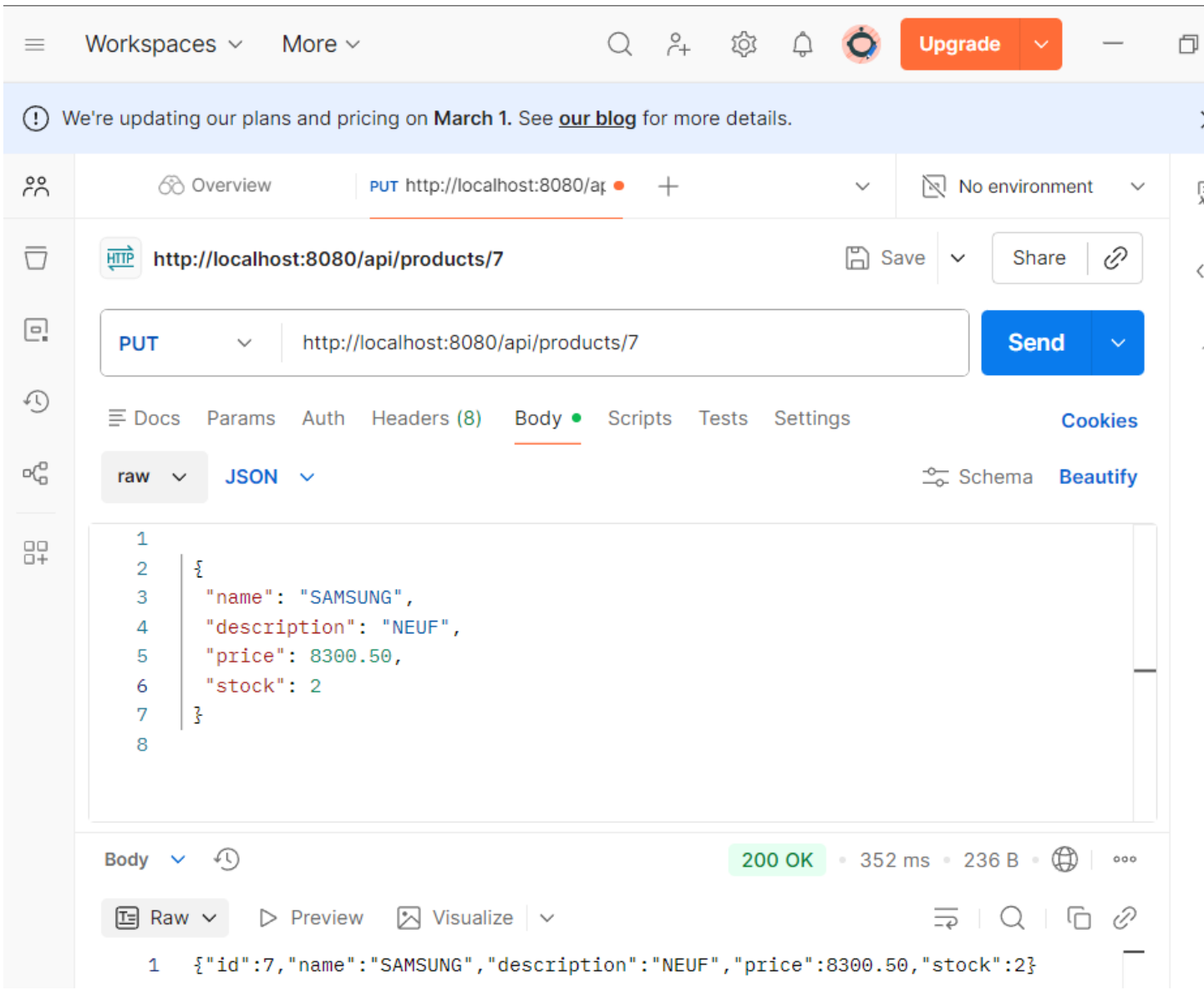

Figure 3 — PUT /api/products/7 : mise à jour (200 OK)

3.4 DELETE — Suppression d'un produit

La requête DELETE sur /api/products/{id} supprime le produit correspondant. La réponse est 204 No Content, confirmant que la suppression a bien eu lieu sans corps de réponse.

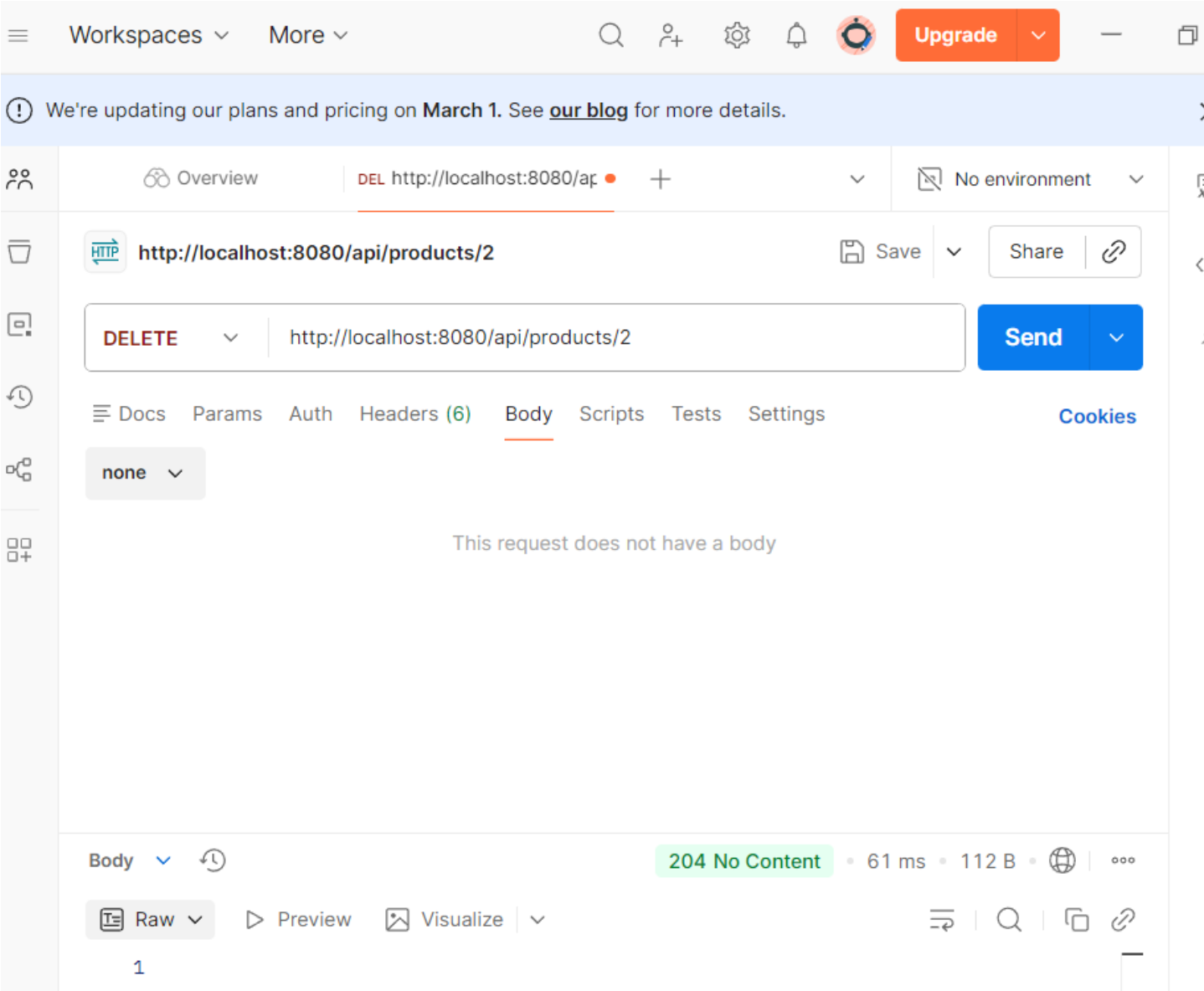

Figure 4 — DELETE /api/products/2 : suppression (204 No Content)

4. Présentation du Code Source

4.1 L'Entité Product

La classe Product est annotée @Entity pour être reconnue par JPA. Elle utilise Lombok pour générer automatiquement les getters, setters et constructeurs, et Jakarta Validation pour contraindre les valeurs saisies.

```
(monolith)   application.properties   Product.java   CategoryRepository.java   ProductRepositor

1   package com.ecommerce.monolith.product.model;
2
3   import jakarta.persistence.*;
4   import jakarta.validation.constraints.*;
5   import lombok.*;
6   import java.math.BigDecimal;
7
8   @Entity // Indique que c'est une table JPA
9   @Table(name = "products") // Nom de la table dans MySQL [cite: 29]
10  @Getter // Génère les Getters via Lombok [cite: 30]
11  @Setter // Génère les Setters via Lombok [cite: 31]
12  @NoArgsConstructor // Constructeur sans argument [cite: 32]
13  @AllArgsConstructor // Constructeur avec tous les arguments [cite: 33]
14  @Builder // Design pattern Builder [cite: 34]
15  public class Product {
16
17      @Id // Clé primaire [cite: 37]
18      @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-incrément pour MySQL [cite
19      private Long id;
20
21      @NotBlank(message = "Product name is required") // Validation : non vide [cite: 41]
22      @Column(nullable = false) // Contrainte SQL : NOT NULL [cite: 42]
23      private String name;
24
25      @Column(length = 1000) // Description longue autorisée [cite: 44, 45]
26      private String description;
27
28      @NotNull(message = "Price is required") // [cite: 46]
```

Figure 5 — Product.java : entité JPA avec annotations Lombok et validation

4.2 Le Repository

L'interface ProductRepository étend JpaRepository et utilise @RepositoryRestResource pour que Spring génère automatiquement tous les endpoints REST CRUD sans écrire de contrôleur. L'annotation @CrossOrigin(*) autorise les appels depuis n'importe quel domaine (utile pour un futur frontend).

CategoryRepository.java | ProductRepository.java | ProductService.java

```java
package com.ecommerce.monolith.product.repository;

import com.ecommerce.monolith.product.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.rest.core.annotation.RepositoryR
import org.springframework.data.repository.query.Param;
import org.springframework.web.bind.annotation.CrossOrigin;
import java.util.List;

// Cette annotation permet à Spring de créer automatiquement une
@RepositoryRestResource(path = "products")  no usages
@CrossOrigin("*") // Utile plus tard si tu veux connecter un fro
public interface ProductRepository extends JpaRepository<Product

    // Version personnalisée pour rechercher un produit par son
    List<Product> findByNameContainingIgnoreCase(@Param("name")
}
```

Figure 6 — ProductRepository.java avec @RepositoryRestResource

4.3 Le Service

ProductService encapsule la logique métier. Le service est annoté @Transactional au niveau de la classe. Les méthodes de lecture portent @Transactional(readOnly = true) pour optimiser les performances. La méthode update() ne nécessite pas d'appel explicite à save() car l'entité est managed par le contexte JPA.

CategoryRepository.java | ProductRepository.java | ProductService.java

```java
package com.ecommerce.monolith.product.service;

import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.repository.ProductReposito
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service // Déclare cette classe comme un Bean de service Spring
@RequiredArgsConstructor // Génère le constructeur pour l'inject
@Transactional // Garantit que chaque opération est une transact
public class ProductService {

    private final ProductRepository repository; // Injection aut

    @Transactional(readOnly = true) // Optimisation pour les lec
    public List<Product> getAll() {
        return repository.findAll(); // Récupère tous les produi
    }

    @Transactional(readOnly = true)  2 usages
    public Product getById(Long id) {
        return repository.findById(id) // Cherche par ID ou lanc
                .orElseThrow(() -> new IllegalArgumentException(
    }

```

Figure 7 — ProductService.java avec gestion des transactions

4.4 Le Contrôleur REST

ProductController expose les 5 endpoints REST sur /api/products. Il utilise @Valid pour déclencher les validations Jakarta sur le corps des requêtes, et retourne des ResponseEntity avec les codes HTTP appropriés (200, 201, 204).

CategoryRepository.java | ProductRepository.java | ProductService.java | ProductController.java

```java
package com.ecommerce.monolith.product.controller;

import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import java.util.List;

@RestController // Déclare une API REST
@RequestMapping("/api/products") // URL de base [cite: 123]
@RequiredArgsConstructor // Injection du service [cite: 124]
@Validated
public class ProductController {

    private final ProductService service;

    @GetMapping // Liste tous les produits [cite: 128]
    public ResponseEntity<List<Product>> getAll() {
        return ResponseEntity.ok(service.getAll());
    }

    @GetMapping("/{id}") // Récupère un produit par son ID [cite: 130, 131]
    public ResponseEntity<Product> getById(@PathVariable Long id) {
        return ResponseEntity.ok(service.getById(id));
    }

    @PostMapping // Crée un nouveau produit [cite: 135, 136]
    public ResponseEntity<Product> create(@Valid @RequestBody Product product) {
```

Figure 8 — ProductController.java : exposition des endpoints REST

5. Exercice : Ajout de l'Entité Category

L'exercice demandé consiste à enrichir l'application par l'ajout d'une entité Category reliée à Product par une relation @ManyToOne / @OneToMany (un produit appartient à une catégorie, une catégorie peut contenir plusieurs produits). L'API de Category est exposée via @RepositoryRestResource.

5.1 L'Entité Category

La classe Category est mappée sur la table "categories". Elle contient un champ name (unique et non nul) et une liste de Product reliée par @OneToMany. L'annotation @JsonIgnore sur la liste évite la boucle infinie lors de la sérialisation JSON.

.java | ProductService.java | ProductController.java | MonolithEcommerceApplication.java | Categor

```java
package com.ecommerce.monolith.product.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import lombok.*;
import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;

@Entity
@Table(name = "categories")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Category name is required")
    @Column(nullable = false, unique = true)
    private String name;

    // Une catégorie peut avoir plusieurs produits
    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL)
    @JsonIgnore // Évite une boucle infinie lors de l'affichage JSON
    private List<Product> products;
}
```

Figure 9 — Category.java : entité avec relation @OneToMany vers Product

Détail de la relation @OneToMany

```java
    // Une catégorie peut avoir plusieurs produits
    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL)
    @JsonIgnore // Évite une boucle infinie lors de l'affichage JSON
    private List<Product> products;
}
```

Figure 10 — Annotation @OneToMany avec cascade ALL et @JsonIgnore

5.2 Le Repository de Category

CategoryRepository étend JpaRepository et est annoté @RepositoryRestResource(path = "categories"). Spring Data REST génère ainsi automatiquement tous les endpoints CRUD pour les catégories, sans contrôleur supplémentaire.

```java
CategoryRepository.java ×   ProductRepository.java   ProductService.java   Pr

package com.ecommerce.monolith.product.repository;

import com.ecommerce.monolith.product.model.Category;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.rest.core.annotation.RepositoryRestResource;

@RepositoryRestResource(path = "categories")  no usages
public interface CategoryRepository extends JpaRepository<Category, Long> {
}
```

Figure 11 — CategoryRepository.java avec @RepositoryRestResource

5.3 Tests Postman — Category

Création d'une catégorie

Un POST sur /categories avec le corps {"name": "SMARTPHONES"} crée la catégorie. La réponse 201 Created inclut les liens HATEOAS générés automatiquement par Spring Data REST.

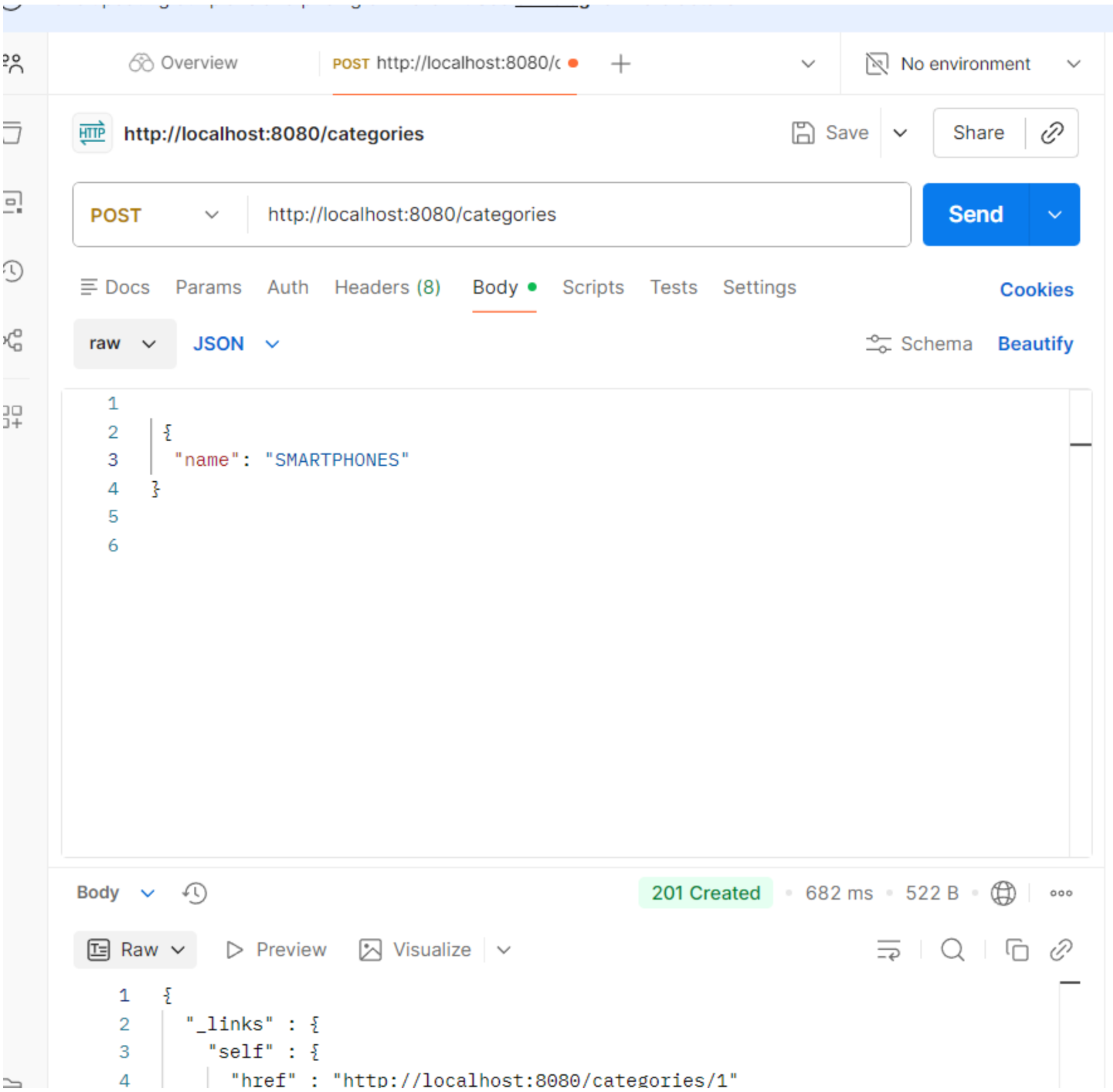

Figure 12 — POST /categories : création de la catégorie SMARTPHONES (201 Created)

Création d'un produit lié à une catégorie

En ajoutant "category": {"id": 1} dans le corps du POST /api/products, le produit est associé à la catégorie créée précédemment. La réponse 201 Created confirme la création avec la relation ManyToOne persistée.

Overview | POST http://localhost:8080/a | No environment

http://localhost:8080/api/products | Save | Share

POST http://localhost:8080/api/products Send

Docs Params Auth Headers (8) Body Scripts Tests Settings Cookie

raw JSON Schema Beautif

```
1
2  {
3
4    "name": "IPHONE 14 PRO MAX",
5    "price": 2300.50,
6    "stock": 5,
7    "category": { "id": 1 }
8
9  }
10
11
```

Body 201 Created 145 ms 281 B

Raw Preview Visualize

```
1  e":"IPHONE 14 PRO MAX","description":null,"price":2300.50,"stock":5,"category":{
```

Figure 13 — POST /api/products : création d'un produit associé à la catégorie (201 Created)

6. Conclusion et Bilan

Ce TP a permis de mettre en pratique les concepts fondamentaux du développement d'applications web avec Spring Boot. L'architecture en couches adoptée garantit une séparation claire des responsabilités et facilite la maintenance du code à long terme.

Compétences acquises

- Initialisation d'un projet Spring Boot via Spring Initializr avec les dépendances adaptées.
- Conception d'une entité JPA avec validations Jakarta et annotations Lombok.
- Utilisation de @RepositoryRestResource pour exposer une API REST sans écrire de contrôleur.
- Implémentation d'un service transactionnel avec gestion des erreurs (orElseThrow).
- Rédaction d'un contrôleur REST complet avec les bonnes pratiques de codes HTTP.
- Modélisation d'une relation bidirectionnelle @ManyToOne / @OneToMany entre deux entités.
- Test de l'ensemble des endpoints à l'aide de Postman.

Avantages et Limites du Monolithe

Avantages	Limites
Simplicité de développement et de déploiement	Scalabilité difficile à grande échelle
Débogage facilité (tout est dans un seul processus)	Couplage fort entre les modules
Bonne performance pour des charges modérées	Un bug peut impacter toute l'application
Idéal pour des équipes petites ou des prototypes	Migration vers les microservices complexe

Ce TP constitue un socle solide pour aborder par la suite les architectures distribuées (microservices), en comprenant d'abord les fondements d'une application monolithique bien structurée.